Final Project Report

Software Design Patterns

Topic: Music Streaming Platform

Students: Nursultan Tumenbay, Abylai Utemis, Nurshakh Sarybek

Source code: https://github.com/Nurswizz/final-project-sdp

1. Introduction

For the final project we had to build one program that uses at least six design patterns from the course, and every student had to implement at least two of them. Our team picked the music streaming service topic from the list. The topic asked for Facade, Observer and Decorator. We added Factory, Strategy and Command because they fit a music player quite naturally, and this way each of us got two patterns.

The idea is a small music player. You can look through a library of tracks, play one, put effects like echo or bass boost on the sound while it plays and watch the lyrics change line by line together with the music. We did not want to use real songs because of copyright, so the program makes the music itself from notes, and we wrote short lyrics for every track ourselves.

Everything is written in Java 17 with Maven and JUnit 5, and we did not use any outside libraries. The sound goes through javax.sound.sampled, which already comes with the JDK, and the web page is served by the small HTTP server that is also part of Java.

2. Project overview

The program has two clients. The first one is a console where you type commands like play t1, add echo or undo. The second one is a small web page that looks like an old pixel mp3 player (section 5). Both clients use the same code in the middle, so we did not have to write the player logic twice. The code is split into packages by what they do:

- model: Track, LyricLine and Lyrics, simple records that only hold data
- library: the track list, the lyrics file parser and the audio library
- audio and audio.effects: the sound stream, the effects and the speaker output
- playback and playback.order: the player and the playback order strategies
- events: the observer interfaces and the listeners
- command and facade: the user actions and MusicStreamingFacade
- app and web: the console app, the web app and the web server

The table shows who was responsible for which pattern. Smaller things like the web page, the tests and this report we did together.

Pattern	Package	Student
Facade	facade	Abylai Utemis
Command	command	Abylai Utemis
Observer	events	Nurshakh Sarybek
Strategy	playback.order	Nurshakh Sarybek
Decorator	audio.effects	Nursultan Tumenbay
Factory	audio.effects	Nursultan Tumenbay

To start the web player, the console version or the tests, run one of these commands in the project folder:

```
./mvnw -q compile exec:java -Dexec.mainClass=music.app.WebApp
./mvnw -q compile exec:java
./mvnw test
```

3. Architecture

When the user presses play, the request actually goes through all six patterns. The client creates a PlayCommand and gives it to CommandHistory. The command calls play() on the facade. The facade finds the track in TrackCatalog, gets the raw sound from AudioLibrary and asks EffectChain to wrap it. EffectChain uses EffectFactory to create one decorator for every active effect. Then the facade gives the wrapped stream to the Player.

The player reads the sound in its own thread, sends it to the speakers and publishes events like TRACK_STARTED and POSITION_CHANGED (about every 100 ms). Listeners such as LyricsDisplay react to these events. When a track ends, the facade asks PlaybackQueue for the next one, and the queue asks the current PlaybackOrder strategy which track that is.

Figure 1 shows the Command, Facade and Observer part, and Figure 2 shows Decorator, Factory and Strategy. We split the diagram in two because one big picture was too hard to read on a page.

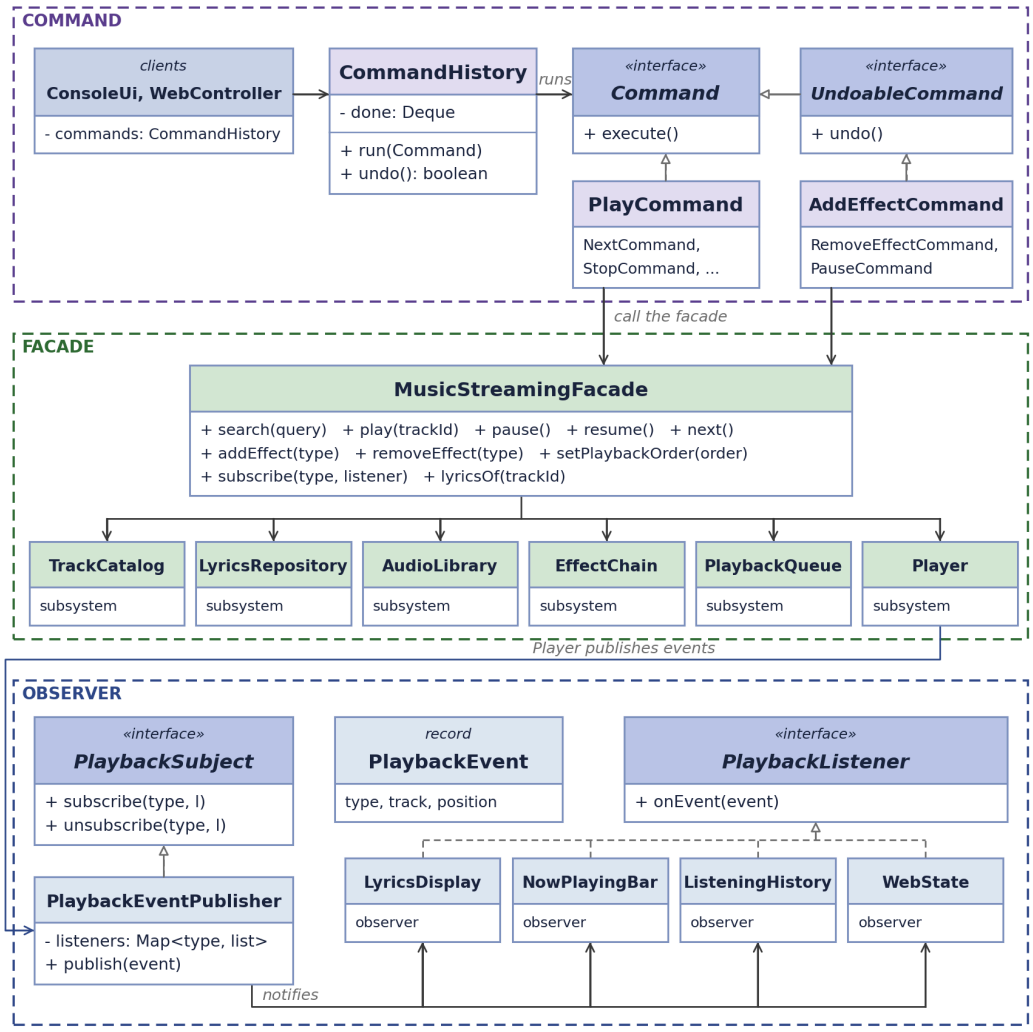


Figure 1. UML class diagram: Command, Facade and Observer

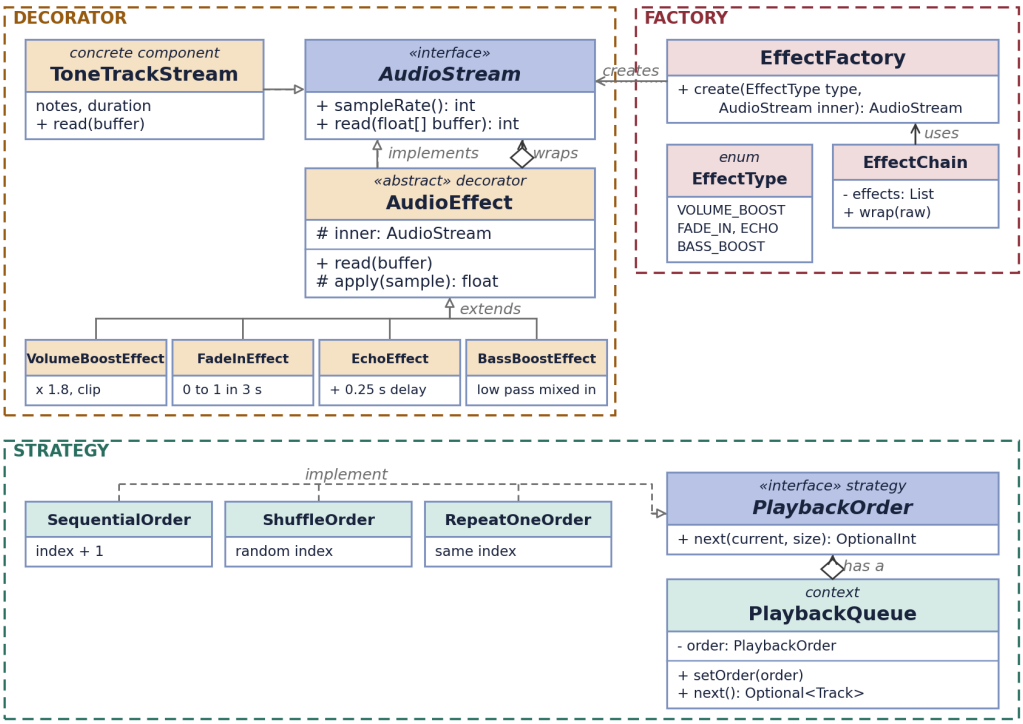


Figure 2. UML class diagram: Decorator, Factory and Strategy

4. Design patterns

4.1 Facade (Abylai Utemis)

Behind the player there are six classes that have to be called in the right order: TrackCatalog, LyricsRepository, AudioLibrary, EffectChain, PlaybackQueue and Player. If the console and the web page called all of them directly, both would repeat the same steps. MusicStreamingFacade hides this, and the clients only see simple methods like play, pause, next and addEffect. The web page was a good test for this, because we added it at the very end and did not have to change the facade at all.

```
public synchronized void play(String trackId) {
    Track track = find(trackId);
    queue.select(track);
    start(track);
}

private void start(Track track) {
    raw = library.open(track);
    player.play(track, effects.wrap(raw));
}
```

4.2 Command (Abylai Utemis)

Every user action is an object: PlayCommand, PauseCommand, AddEffectCommand and so on. CommandHistory runs a command and keeps the ones that can be undone on a stack. We added a separate UndoableCommand interface, because undo makes sense only for some actions. Undoing an added effect removes it and undoing a pause resumes the music. We could not think of a good meaning for undoing play or next, so those commands just do not go to the history.

```
public void run(Command command) {
    command.execute();
    if (command instanceof UndoableCommand undoable) {
        done.push(undoable);
    }
}
```

4.3 Observer (Nurshakh Sarybek)

The player should not know who shows the lyrics or the progress bar. It only publishes PlaybackEvent objects through PlaybackEventPublisher, and listeners subscribe to the event types they need. LyricsDisplay listens to POSITION_CHANGED and prints the lyric line whose time has come, NowPlayingBar draws the progress, ListeningHistory remembers finished tracks and WebState keeps the state for the web page. We used the push model, so every event already carries the track and the position. Because the player runs in its own thread, the listener lists are a ConcurrentHashMap of CopyOnWriteArrayList, and one broken listener cannot stop the music.

```
public void publish(PlaybackEvent event) {
    for (PlaybackListener listener : listenersOf(event.type())) {
        try {
            listener.onEvent(event);
        } catch (RuntimeException e) {
            System.err.println("Listener failed on " + event.type() + ": " + e);
        }
    }
}
```

4.4 Strategy (Nurshakh Sarybek)

The next track can be chosen in different ways: in order, at random, or the same track again. Each way is a class that implements PlaybackOrder with one method, next(current, size). PlaybackQueue is the context. It keeps the tracks and asks the current strategy which index comes next, and the user can switch the order while the music is playing. ShuffleOrder gets its Random object through the constructor, so in tests we pass a fixed seed and get the same result every time.

```
public Optional<Track> next() {
    OptionalInt index = order.next(current, tracks.size());
    if (index.isEmpty()) {
        return Optional.empty();
    }
    current = index.getAsInt();
    return Optional.of(tracks.get(current));
}
```

4.5 Decorator (Nursultan Tumenbay)

This pattern was the most fun one, because you can actually hear it. AudioStream has only two methods, sampleRate() and read(buffer). ToneTrackStream makes the music, and every effect is a decorator that wraps another AudioStream, reads samples from it and changes them. Since an effect is also an AudioStream, effects can be stacked in any order, for example new EchoEffect(new BassBoostEffect(raw)), and the player never knows how many effects are inside. When the user adds an effect during playback, the facade wraps the raw stream again and the player switches to the new chain on the next buffer.

```
public int read(float[] buffer) {
    int count = inner.read(buffer);
    for (int i = 0; i < count; i++) {
        buffer[i] = apply(buffer[i]);
    }
    return count;
}
```

4.6 Factory (Nursultan Tumenbay)

The commands and both clients only know the EffectType enum: VOLUME_BOOST, FADE_IN, ECHO and BASS_BOOST. EffectFactory turns a type into the right decorator, and EffectChain goes through the active types and calls the factory for each of them. This way EffectChain does not depend on the concrete effect classes, and a new effect needs one new class, one enum value and one more line in the factory.

```
public AudioStream create(EffectType type, AudioStream inner) {
    return switch (type) {
        case VOLUME_BOOST -> new VolumeBoostEffect(inner);
        case FADE_IN -> new FadeInEffect(inner);
        case ECHO -> new EchoEffect(inner);
        case BASS_BOOST -> new BassBoostEffect(inner);
    };
}
```

5. Web player and screenshots

At the end we felt that a console alone looks a bit dry for a music app, so we made a small web page. It is one HTML file with a canvas where everything is drawn pixel by pixel, even the letters, on a plain white background. The click wheel has menu, previous, next and play/pause, and the six keys under it switch the four effects, change the playback order and undo the last action. On the Java side there are three new classes. WebServer uses the HttpServer from the JDK, WebController turns clicks into the same Command objects as the console, and WebState is one more Observer listener that keeps the current track, position and lyric line. The page asks the server for this state four times a second. The sound itself still plays from Java on the computer, so the page works like a remote control.



Figure 3. The web player: playing with echo and bass boost (left), the library menu (middle), paused in shuffle mode (right)

6. Testing

The project has 46 JUnit 5 tests and all of them pass with ./mvnw test. They check that VolumeBoostEffect makes the sound louder and clips it, that FadeInEffect starts silent and that EchoEffect repeats the sound later. Other tests cover undo in reverse order, the event publisher (including a listener that unsubscribes itself while handling an event), every listener, the lyrics parser, the three playback orders, the facade playing a whole queue and the web controller. In the tests the player writes to SilentOutput instead of the speakers, so they run fast and do not make any noise.

7. Conclusion

The project shows that six patterns can live in one small program without getting in each other's way. The facade became the center of the design, because both clients and all commands talk only to it. Observer made the lyrics easy, since the player does not care who is listening. Decorator and Factory together made the effects simple to add and combine, and Strategy and Command were small but useful for shuffle and undo.

The hardest part for us was working with threads. The player plays on its own thread, and a listener cannot start the next track on that same thread, because the player would end up waiting for itself. That is why the facade starts the next track on a separate single thread executor. We also learned that agreeing on the interfaces first (AudioStream, PlaybackListener and the facade methods) let the three of us work at the same time without waiting for each other.

8. Further work

- play real audio files, for example WAV, through an adapter, so the library is not only generated music
- save playlists and the listening history to a file or a small database
- add a seek bar and a volume slider to the web player
- send events to the page with server-sent events instead of asking four times a second
- add an equalizer as one more decorator

References

- Gamma, E., Helm, R., Johnson, R., Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994
- Freeman, E., Robson, E. Head First Design Patterns, 2nd edition. O'Reilly, 2020
- Refactoring.Guru: Design Patterns, https://refactoring.guru/design-patterns
- Oracle Java SE 17 documentation: javax.sound.sampled and com.sun.net.httpserver packages